

Unity/Ceedling 单元测试接入方案（APP 固件）

- 目标：对 APP 固件的**纯逻辑层**（协议解析/状态机/CRC/命令处理/OTA 判定等）
- 建立主机单元测试回归防线；硬件依赖用 CMock 打桩。
- 边界：**硬件时序/中断/FSMC 等测不了**（本次白屏/死机即此类——需 HIL 层）。

1. 框架选型

依赖：**Ruby**（Windows 安装 RubyInstaller，或 CI 用 `gem install ceedling`）。

备选（无 Ruby 环境）：仅 Unity + 现有 CMake/脚本构建（功能等价，mock 手写）。

2. 目录结构（APP/APP 内新增）

```
APP/APP/
├── tests/
│   ├── unity/                # 框架源码 (vendor: unity.c/h, cmock.c/h)
│   ├── project.yml           # Ceedling 配置 (单一事实源)
│   └── test/                  # 测试用例 test_*.c
│       ├── test_cam_link_parser.c # 摄像头帧协议状态机 (本次排查模块)
│       ├── test_event_bus.c      # 事件总线订阅/发布
│       ├── test_ota_replay.c     # OTA 防重放判定 (build_no 递增)
│       ├── test_wdog_stall.c     # 任务看门狗超时判定
│       ├── test_cmd_catalog.c    # 命令解析/分发
│       ├── test_logger_fmt.c     # 日志缓冲/格式化
│       └── test_crc_frame.c      # 帧校验和/CRC
│   ├── mocks/                # CMock 生成的 mock_*.c/h (build 产物, 不入库)
│   ├── support/               # 测试桩/辅助 (stub_*.c)
│   └── build/                  # Ceedling 构建输出 (不入库)
└── tests/... (现有 ctest 体系保留, 二者共存)
```

3. 环境准备

```
# Windows (一次)
winget install RubyInstallerTeam.RubyWithDevKit # 或手动装 RubyInstaller
gem install ceedling
```

```
# CI (.github/workflows/ci.yml 新增 job)
# - name: Unit tests (Unity/Ceedling)
#   run: |
#     gem install ceedling
#     cd APP/APP && ceedling test:all
```

4. project.yml 配置

```
:project:
  :use_exceptions: FALSE
  :use_test_preprocessor: TRUE
  :use_auxiliary_dependencies: TRUE
  :build_root: build/ceedling
  :test_file_prefix: test_
  :which_ceedling: gem

:unity:
  :defines:
    - UNITY_SUPPORT_64

:cmock:
```

```

:mock_prefix: mock_
:enforce_strict_ordering: TRUE
:treat_extens: :include
:includes_h:
- stdint.h
- stdbool.h

:paths:
:test:      [ tests/unity/test/ ]
:source:    [ SystemServices/, Application/, Config/, BSP/ ]
:support:   [ tests/unity/support/ ]
:include:   [ Config/, Core/Inc/, BSP/ ]

:defines:
- UNIT_TESTING
- APP_DEBUG_MODE=0

:flags:
:test:
:compile:
:*:
- -std=c99
- -Wall
- -Werror

```

5. 测试用例模板（cam_link 帧解析——本次死机排查的模块）

```

/* test_cam_link_parser.c */
#include "unity.h"
#include "cam_link.h"

void setUp(void) { CamLink_Init(); }
void tearDown(void) {}

/* 手部坐标帧: AA 55 | 0x01 | LEN=9 | flags, x, y, w, h(各2B) | SUM */
void test_hand_frame_parses_and_sets_state(void)
{
    uint8_t f[] = { 0xAA, 0x55, 0x01, 0x09,
                    0x01, 0x10, 0x00, 0x20, 0x00, 0x30, 0x00, 0x40, 0x00,
                    0x00 /*SUM*/ };
    for (unsigned i = 0; i < sizeof(f); i++) CamLink_OnRxByte(f[i]);
    TEST_ASSERT_EQUAL_UINT32(1, CamLink_GetState()->frame_count);
    TEST_ASSERT_TRUE(CamLink_GetState()->hand_present);
    TEST_ASSERT_EQUAL_UINT16(0x10, CamLink_GetState()->hand_x);
    TEST_ASSERT_EQUAL_UINT16(0x20, CamLink_GetState()->hand_y);
}

/* 校验和错误: 帧丢弃并计数 */
void test_bad_checksum_counts_error(void)
{
    uint8_t f[] = { 0xAA, 0x55, 0x01, 0x09,
                    0x01, 0x10, 0x00, 0x20, 0x00, 0x30, 0x00, 0x40, 0x00,
                    0xFF /*错 SUM*/ };
    for (unsigned i = 0; i < sizeof(f); i++) CamLink_OnRxByte(f[i]);
    TEST_ASSERT_EQUAL_UINT32(0, CamLink_GetState()->frame_count);
    TEST_ASSERT_EQUAL_UINT32(1, CamLink_GetState()->err_count);
}

/* 失步恢复: 垃圾字节后仍能收到合法帧 */
void test_resync_after_garbage(void)
{
    uint8_t g[] = { 0x00, 0x11, 0xAA, 0x22, 0xAA, 0xAA };
    for (unsigned i = 0; i < sizeof(g); i++) CamLink_OnRxByte(g[i]);
    /* ... 正常帧 ... */
    TEST_ASSERT_EQUAL_UINT32(1, CamLink_GetState()->frame_count);
}

```

6. CMock 桩示例（硬件依赖打桩）

```
/* test_cam_link_dispatch.c —— 帧分发依赖 BSP_GetTick（硬件 tick）*/
#include "unity.h"
#include "cam_link.h"
#include "mock_bsp_system.h" /* CMock 自动生成 */

void test_frame_records_rx_time(void)
{
    BSP_GetTick_ExpectAndReturn(0x1234);
    /* 喂一帧合法帧 */
    /* ... */
    TEST_ASSERT_EQUAL_UINT32(0x1234, CamLink_GetState()->last_rx_ms);
}
```

7. 运行

```
cd APP/APP
ceedling test:all          # 全量
ceedling test:cam_link    # 单模块
ceedling clobber           # 清理
```

8. 与现有体系共存（不破坏已有测试）

9. 建议首批测试模块（高价值 + 纯逻辑）

1. ****cam_link 帧解析状态机****（AA55 协议——本次排查核心，逻辑纯度高）
2. ****event_bus 订阅/发布/消费****（消息总线——全系统依赖）
3. ****OTA 防重放判定****（build_no 递增——本次 OTA 失败根因的回归防线）
4. ****WDOG 超时判定****（任务看门狗逻辑）
5. ****命令解析****（cmd_catalog——shell 命令分发）
6. ****帧校验和****（协议 SUM/CRC）

10. 明确边界（务必知悉）

- ☐ ****单测覆盖****：语法、纯逻辑、状态机、数据处理、边界条件、回归
- ☐ ****单测不覆盖****：硬件时序（FSMC/LCD）、中断响应、DMA、电源/EMI、

并发竞态、真实外设交互——****这些需要 HIL（真板）+ 长稳测试****——

本次白屏/死机即硬件时序问题，单测****发现不了****，但单测能防止

****同类逻辑错误回归****（如协议解析、防重放、命令处理）。

组件	作用	说明
---	---	---
Unity	断言 + 测试框架	嵌入式 C 事实标准，单文件头+实现
CMock	自动生成 C 函数 mock	为硬件依赖（BSP_*/HAL_*）生成桩
Ceedling	构建/运行/生成器	Ruby 工具，管理编译、测试、mock 生成